

mettā muse — Full-Stack E-Commerce PLP

Complete Architecture, Frontend/Backend Deep-Dive, Defensive Safeguards, and 1-Line Interview Cheat Sheet

REPOSITORY	AUTHOR	DEPLOYMENTS	TEST COVERAGE
Appscrip-task-AdityaNayak	Aditya Nayak	Netlify Edge + Render + Neon PostgreSQL	108 Passing Tests (Unit + E2E), 0 Warnings

1. MONOREPO ARCHITECTURE OVERVIEW

The application is an enterprise-grade e-commerce monorepo separating server-side rendering, client interactive islands, modular REST backend services, and relational database persistence.

1. CLIENT VIEWPORT (DESKTOP 1440PX | TABLET 768PX | MOBILE 375PX)

Pure CSS Modules (Zero UI Kits) • Semantic HTML5 • Responsive Drawer • Accessible ARIA Attributes

▼ User Action (Filter Checkbox, Sort Dropdown, Search Input, Pagination Link, Wishlist Toggle)

2. NEXT.JS 16 APP ROUTER (NETLIFY EDGE RUNTIME)

Server Component (ProductsPage SSR) • Client Islands (useTransition, usePlpNavigation) • Server-Side apiFetch (35s timeout)

▼ HTTP GET /products?category=...&sort=...&page=... (Server-to-Server Fetch)

3. NESTJS 12 REST API (RENDER CLUSTER)

ProductsController (OpenAPI Swagger at /docs) • ListProductsQueryDto (class-validator pipes) • ProductsService • Deterministic Sort

▼ Type-Safe Prisma ORM Queries (findMany, count, where, orderBy)

4. POSTGRESQL 16 DATABASE (NEON AWS REGION)

Product & Category Relational Models • @@index([categoryId, price]) • @@index([price]) • pg_trgm GIN Trigram Search Index

2. "WHY X?" — THE 1-LINE TECHNICAL FOUNDATIONS

These one-line answers explain the rationale behind every core technology choice:

TECHNOLOGY	WHY WE CHOSE IT (WHAT TO TELL THE INTERVIEWER)
React 19	Allows splitting the UI into clean, reusable components (cards, headers, filters) with modern Server Component / Client Island capabilities.
Next.js 16 App Router	Provides genuine Server-Side Rendering (SSR) out of the box, generating real HTML on the server for instant Google crawler discoverability.
NestJS 12	Delivers enterprise-grade architecture with modular controllers, dependency injection, automated Swagger docs, and strict DTO validation pipes.
TypeScript	Guarantees end-to-end compile-time type safety across database schemas, API DTO contracts, and React component props, preventing runtime crashes.
Prisma ORM	Writes type-safe PostgreSQL queries, manages automated schema migrations, and eliminates fragile raw SQL strings.
PostgreSQL 16	An ACID-compliant relational database with advanced indexing support (B-tree composite indexes and pg_trgm GIN trigram fuzzy search).
Pure CSS Modules	Matches Figma with 100% pixel fidelity while eliminating UI kit bloat (no Tailwind/Bootstrap/MUI), generating 0kb JavaScript runtime overhead.

3. FRONTEND ARCHITECTURE: WHAT WE USE & WHY

The frontend follows the **Client Island Pattern**. The heavy visual shell renders on the server with Okb JavaScript, while only interactive elements hydrate on the client.

1. Next.js 16 App Router & Server Components (SSR)

File: frontend/src/app/products/page.tsx

How it works: The page component `ProductsPage` is a React Server Component. It fetches data from our NestJS backend on the server during request time. It generates dynamic metadata (`generateMetadata`) and injects Schema.org JSON-LD structured data into the HTML `<head>` .

What to say in interview: *"We use Next.js Server Components for mandatory SSR. When Googlebot or curl inspects the raw HTML, all 18 product titles, prices, and the H1 are already in the document before any JavaScript executes in the browser."*

2. Client Islands ('use client') for Dynamic Interactions

Files: `SortDropdown.tsx` , `FilterSidebar.tsx` , `FilterToggle.tsx` , `WishlistButton.tsx` , `MobileMenu.tsx`

How it works: We strictly limit 'use client' to components that require browser event listeners (clicks, onChange) or React hooks. The parent page and product grid remain Server Components.

What to say in interview: *"We don't make the entire page a client component. We isolate client interactivity into small islands. This keeps our client-side bundle tiny and optimizes Core Web Vitals."*

3. URL State Synchronization via React 19 useTransition

File: frontend/src/lib/url/use-plp-navigation.ts

How it works:

```
const [isPending, startTransition] = useTransition();
startTransition(() => {
  router.push(href, { scroll: false });
});
```

The browser URL acts as the single source of truth. When a user clicks a filter, `startTransition` tells Next.js to fetch the new Server Component payload in the background and smoothly reconcile the DOM without reloading the page or jumping scroll position.

What to say in interview: *"All filter, sort, and pagination state lives in the URL. We use React 19's useTransition so filtering is non-blocking, shareable, and bookmarkable without full-page refreshes."*

4. Hydration-Safe Wishlist via useSyncExternalStore

Files: frontend/src/components/plp/WishlistButton.tsx & frontend/src/lib/wishlist/store.ts

How it works: Reading `localStorage` inside `useEffect` causes React 19 hydration mismatch errors because server HTML differs from client storage. We use `useSyncExternalStore` to subscribe directly to browser storage events.

What to say in interview: *"Instead of buggy setState inside useEffect, useSyncExternalStore guarantees instant optimistic heart fills (#EB4C6B) with zero hydration flashes and zero network lag."*

5. Responsive Viewport Strategy (Desktop, Tablet, Mobile)

How it works: • **Desktop (1440px):** 4-column product grid with 300×462px cards and an inline left filter sidebar.

• **Custom Tablet (768px):** 3-column grid with a responsive collapsible filter bar.

• **Mobile (375px):** 2-column grid with a slide-out hamburger drawer using the native HTML `<dialog>` element with focus trapping.

What to say in interview: *"Figma provided desktop and mobile frames. We implemented them with pixel precision and added a custom tablet layout at 768px, verified across 8 viewports with 0 horizontal overflow."*

4. FRONTEND DEFENSIVE CHECKS & SAFETY GUARDS

The frontend is hardened with 5 distinct defensive layers to prevent runtime errors, malicious input crashes, and cloud network latency issues.

🔒 Check 1: Input & URL Sanitization (parsePlpSearchParams)

File: `frontend/src/lib/url/plp-params.ts`

The Problem: Users or bots can type invalid queries like `?page=-5` or `?sort=drop_table`.

The Defense: • **Page:** Parsed as integer `$ge 1$`; defaults to `1`.

• **Sort:** Validated against a strict whitelist `['recommended', 'newest', 'popular', 'price_asc', 'price_desc']`.

• **Category:** Regex-checked against `/^[a-z0-9-]+$` (max 10 slugs). Unrecognized slugs never reach the database.

What to say in interview: *"Our URL parser is completely defensive. It normalizes all input before passing it to the fetch layer, ensuring the application never throws unhandled exceptions on corrupt URLs."*

🔒 Check 2: Out-of-Bounds Pagination Redirect (HTTP 308)

File: `frontend/src/app/products/page.tsx` (Lines 62–67)

The Problem: A user clicks a bookmark for `?page=99` on a category that now only has 2 pages.

The Defense: The server checks `if (query.page > paginatedProducts.meta.totalPages)` and issues an immediate permanent redirect (HTTP 308) back to the last valid page. Users never see an empty broken screen.

🔒 Check 3: Free-Tier Cold Start Tolerance (Timeout & Retry)

File: `frontend/src/lib/api/client.ts`

The Problem: Render free-tier APIs spin down after 15 minutes of inactivity, taking ~30 seconds to wake.

The Defense: Standard fetches timeout after 10s. Our `apiFetch` client sets an `AbortSignal.timeout(35000)` (35-second threshold) and includes an automatic retry after 2000ms before bubbling up a graceful error boundary.

🔒 Check 4: Security Boundary Enforcement (import 'server-only')

Files: `frontend/src/lib/api/client.ts` & `frontend/src/lib/api/products.ts`

The Problem: A developer might accidentally import a server API client into a Client Component, exposing private API URLs or environment keys in client JavaScript bundles.

The Defense: Adding `import 'server-only'` triggers a compile-time build failure if the file is imported anywhere in the client tree.

🔒 Check 5: Accessibility & WCAG 2.1 AA Compliance

The Problem: Figma's body text color token `#888792` has a 3.54:1 contrast ratio on white, failing WCAG AA (requires 4.5:1).

The Defense: We overrode body copy with `#6B6A75`, achieving a 5.33:1 contrast ratio. Additionally, mobile drawer navigation traps focus inside the native `<dialog>` element, and all links use explicit `aria-label` and `aria-current="page"` attributes.

Is every line needed in the frontend?

Yes. On October 4th, we completed an automated audit using `knip` and TypeScript strict checks. We removed 15 redundant `import React` statements (not needed in React 19's JSX runtime), dropped 2 dead barrel files, and cleaned up unused CSS classes. Every line remaining serves a direct functional or defensive purpose.

5. BACKEND ARCHITECTURE: NESTJS REST API

The backend is built with **NestJS 12**, providing enterprise modularity, dependency injection, automated OpenAPI Swagger documentation, and strict DTO validation pipes.

1. NestJS Architecture Layers (Controllers, Services, DTOs, Modules)

- **Controllers:** Handle HTTP routing, status codes, and OpenAPI Swagger decorators.
- **DTOs (Data Transfer Objects):** Validate and transform incoming request payloads using `class-validator`.
- **Services:** Execute business logic, compute pagination math, and invoke Prisma database queries.
- **Modules:** Encapsulate related features (`ProductsModule` , `CategoriesModule` , `HealthModule`).

What to say in interview: *"NestJS provides clear separation of concerns. Controllers handle HTTP contracts, DTOs validate incoming data, and Services manage business logic and database access via Prisma."*

2. ProductsController (HTTP Endpoints & Swagger)

File: `backend/src/products/products.controller.ts`

- `GET /products` : Supports pagination (`page` , `limit`), category filter, price bounds (`minPrice` , `maxPrice`), sort order, and search (`q`).
- `GET /products/:id` : Returns a single product with full gallery images.
- Fully decorated with `@ApiTags` , `@ApiOperation` , and `@ApiResponse` , generating live Swagger UI at `/docs` .

3. ListProductsQueryDto (The Input Security Gatekeeper)

File: `backend/src/products/dto/list-products-query.dto.ts`

Uses declarative `class-validator` decorators:

```
@IsOptional()
@IsInt()
@Min(1)
page: number = 1;

@IsOptional()
@IsInt()
@Max(48)
limit: number = 18; // Default 18 matches Figma 6 rows x 3 cols

@IsOptional()
@Transform(({ value }) => typeof value === 'string' ? value.split(',') : value)
@IsArray()
category?: string[];
```

Includes a custom validator ensuring `minPrice ≤ maxPrice` . Any illegal input immediately returns a `400 Bad Request` .

4. ProductsService & Pagination Logic

File: `backend/src/products/products.service.ts`

Executes query orchestration. Calculates SQL offsets: `const skip = (page - 1) * limit;` . Runs product retrieval and total counting concurrently:

```
const [items, total] = await Promise.all([
  this.prisma.product.findMany({ skip, take: limit, where, orderBy }),
  this.prisma.product.count({ where }),
]);
const totalPages = Math.ceil(total / limit);
```

5. Deterministic Sort Strategies (products.sort.ts)

File: `backend/src/products/products.sort.ts`

Maps sort keys to Prisma clauses. Crucially adds a secondary tie-breaker on `id` : • `price_asc` : `[{ price: 'asc' }, { id: 'asc' }]`

- `price_desc` : `[{ price: 'desc' }, { id: 'desc' }]`

What to say in interview: *"If multiple products share the exact same price, sorting without a secondary tie-breaker causes items to drift between pages. Adding id guarantees pagination determinism."*

6. DATABASE ARCHITECTURE: POSTGRESQL & PRISMA ORM

The persistence layer uses **PostgreSQL 16** hosted on Neon AWS, managed through **Prisma ORM** with composite indexing and trigram search.

1. Relational Database Schema (schema.prisma)

```
model Category {
  id      Int      @id @default(autoincrement())
  slug    String   @unique
  name    String
  products Product[]
}

model Product {
  id          Int          @id @default(autoincrement())
  slug        String       @unique
  title       String
  description  String
  price       Decimal      @db.Decimal(10, 2)
  rating      Float        @default(0)
  ratingCount Int          @default(0)
  categoryId  Int
  category    Category     @relation(fields: [categoryId], references: [id])
  images      ProductImage[]
  createdAt   DateTime      @default(now())

  @@index([categoryId, price]) // Composite index
  @@index([price])             // Single price sort
  @@index([createdAt])         // Newest sort
  @@index([ratingCount])       // Popularity sort
}
```

2. The Composite Index Strategy: @@index([categoryId, price])

The Problem: In e-commerce, the most frequent query is filtering by category and sorting by price simultaneously (e.g. "Women's Clothing sorted by Price"). Without an index, PostgreSQL must scan every row in that category and sort in memory.

The Solution: Our composite index `@@index([categoryId, price])` creates a multi-column B-tree structure. PostgreSQL jumps directly to the category slice and reads rows already sorted by price in single-digit milliseconds.

What to say in interview: "A composite index on (categoryId, price) avoids expensive in-memory sort operations, speeding up our most common e-commerce catalog query by 10x."

3. PostgreSQL pg_trgm Trigram Search Index

Migration: `backend/prisma/migrations/20261003112228_init_pg_trgm/migration.sql`

How it works: Standard SQL `LIKE '%query%'` cannot use normal B-tree indexes because of leading wildcards. In our migration, we enabled PostgreSQL's native `pg_trgm` extension and built a GIN trigram index on `(title, description)`. It breaks words into 3-character tokens, allowing fast sub-string fuzzy matching.

4. PrismaService Offline Resilient Store Fallback

File: `backend/src/prisma/prisma.service.ts`

During application startup (`onModuleInit`), if PostgreSQL is unreachable due to a network drop, it catches the error and initializes a zero-dependency in-memory store populated with the 60-item catalog snapshot. Local unit and E2E tests pass smoothly without needing a live Docker container.

5. Standardized Error Envelope (http-exception.filter.ts)

File: `backend/src/common/filters/http-exception.filter.ts`

Intercepts all thrown exceptions and standardizes them into an RFC-compliant response:

```
{
  "statusCode": 400,
  "message": "Validation failed",
  "error": "Bad Request",
  "timestamp": "2026-10-05T04:12:00.000Z",
  "path": "/products"
}
```

7. THE 7 CORE END-TO-END FUNCTION CALL FLOWS

These step-by-step flows show how data travels from user click, through the network, into the database, and back to the screen.

Flow 1: Initial SSR Page Load (The Crawler Discoverability Flow)

User visits `/products` → `ProductsPage(searchParams)` in `page.tsx` → `parsePlpSearchParams()` sanitizes query → `getProducts()` calls `apiFetch('/products')` on server → NestJS `ProductsController.findAll()` validates `ListProductsQueryDto` → `ProductsService.findAll()` executes `prisma.product.findMany({ skip: 0, take: 18 })` → PostgreSQL executes query → Raw HTML containing all 18 product titles, prices, and JSON-LD is streamed to the browser → Client islands hydrate.

Flow 2: Category Filter Change (The Seamless Transition Flow)

User clicks "Electronics" checkbox in `FilterSidebar.tsx` → `handleCategoryChange('electronics')` fires → `navigate({ category: ['electronics'], page: 1 })` in `usePlpNavigation` → `buildPlpHref()` formats URL → React 19 `startTransition(() ⇒ router.push(href, { scroll: false }))` → Next.js App Router re-renders Server Component in background → NestJS queries `where: { category: { slug: { in: ['electronics'] } } }` using composite index → Updated cards stream into DOM → Count changes to "15 Items", no full page reload.

Flow 3: Sort Order Strategy Execution

User selects "Price: High to Low" in `SortDropdown.tsx` → `handleSelect('price_desc')` → Pushes `?sort=price_desc` via `usePlpNavigation` → Backend `products.sort.ts` maps to `[{ price: 'desc' }, { id: 'desc' }]` → Prisma uses single index `@@index([price])` → Returns products ordered from highest price (\$99.99 Amazon Echo Plus) to lowest.

Flow 4: Optimistic Wishlist Button Toggle

User clicks heart icon on a card → `WishlistButton.handleToggle(e)` executes `e.stopPropagation()` → Calls `toggleWishlist(productId)` in `store.ts` → Reads/writes to `localStorage` → Emits custom window event → `useSyncExternalStore` triggers reactive re-render → Heart fills red (`#EB4C6B`) and `aria-pressed="true"` . Oms network latency.

Flow 5: Search & Trigram Fuzzy Text Query

User visits `/products?q=jacket` → `ListProductsQueryDto` validates length \$1e 80\$ → `ProductsService` builds `where.OR = [{ title: contains }, { description: contains }]` → PostgreSQL GIN trigram index evaluates fuzzy match → If 0 matches, `EmptyState.tsx` renders with a "Clear all filters" CTA.

Flow 6: Numbered Pagination Navigation

User clicks Page 2 in `Pagination.tsx` → Pushes `?page=2` → Backend computes `skip = (2 - 1) * 18 = 18` , `take = 18` → Returns page 2 products → If user visits `?page=999` , `page.tsx` issues an automatic 308 redirect back to the last valid page.

Flow 7: Network Resilience & Cold Start Retry

Render free-tier API sleeping → `apiFetch` in `client.ts` initiates fetch with 35s timeout → If initial attempt fails, catches error, waits 2000ms, and retries automatically → If backend DB fails, `PrismaService` falls back to in-memory store, preventing server crashes.

8. FILE-TO-FILE CONNECTIVITY MATRIX

This table maps every core file, its architectural runtime layer, imported dependencies, and caller components:

FILE PATH	RUNTIME LAYER	DEPENDENCIES / IMPORTS	CALLERS / DEPENDENTS
frontend/src/app/products/page.tsx	Server Component (SSR)	@/lib/api , @/lib/url , @/components/plp , @/components/seo/JsonLd	Next.js App Router endpoint for /products & /shop
frontend/src/lib/api/client.ts	Server Only (server-only)	Node fetch , AbortSignal , ./errors	getProducts , getCategories
frontend/src/lib/url/use-plp-navigation.ts	Client Island ('use-client')	next/navigation , react (useTransition) , ./plp-params	FilterSidebar , SortDropdown , Pagination , EmptyState
frontend/src/components/plp/ProductGrid.tsx	Server Component (SSR)	./ProductCard	ProductsPage
frontend/src/components/plp/ProductCard.tsx	Server Component (SSR)	next/image , ./WishlistButton	ProductGrid
frontend/src/components/plp/WishlistButton.tsx	Client Island ('use-client')	react (useSyncExternalStore) , @/lib/wishlist/store	ProductCard
frontend/src/components/plp/FilterSidebar.tsx	Client Island ('use-client')	@/lib/url/use-plp-navigation , @/components/ui/icons	ProductsPage
frontend/src/components/plp/SortDropdown.tsx	Client Island ('use-client')	@/lib/url/use-plp-navigation , @/components/ui/icons	Toolbar
frontend/src/components/plp/Pagination.tsx	Client Island ('use-client')	@/lib/url/use-plp-navigation , @/lib/url/pagination-items	ProductsPage
backend/src/main.ts	Node / NestJS Entrypoint	@nestjs/core , @nestjs/swagger , ./app.module	Render start command: node dist/main.js
backend/src/products/products.controller.ts	NestJS REST Controller	@nestjs/common , ./products.service , ./dto/*	Incoming HTTP requests at /products
backend/src/products/products.service.ts	NestJS Injectable Service	../prisma/prisma.service , ./products.sort	ProductsController.findAll & findById
backend/src/prisma/prisma.service.ts	Prisma ORM Client	@prisma/client , ../common/utils/slugify	ProductsService , CategoriesService

9. RAPID-FIRE TECHNICAL INTERVIEW Q&A (1-LINE ANSWERS)

Direct, confident one-line responses to the most likely questions interviewers will ask:

Q1 How does your Server-Side Rendering (SSR) work?

👉 "The page is a React Server Component that fetches our NestJS API on the server and streams raw HTML with product titles and Schema.org markup directly to the browser."

Q2 Why didn't you use Tailwind CSS or a UI kit?

👉 "To achieve 100% pixel fidelity to Figma without fighting library overrides, and to eliminate runtime JavaScript bloat, achieving a 100/100 Lighthouse desktop score."

Q3 How do filters update without a full page reload?

👉 "We update query parameters using React 19's `useTransition`, which prompts Next.js to fetch only the updated Server Component subtree in the background."

Q4 Why did you use `useSyncExternalStore` for the wishlist?

👉 "To safely subscribe to `localStorage` without causing React 19 hydration mismatch warnings between server HTML and client state."

Q5 How did you make the layout responsive?

👉 "We built a 4-column desktop grid with an inline sidebar that smoothly transitions to a 2-column mobile grid with an accessible slide-out drawer below 1200px."

Q6 What is a DTO and why do you use it?

👉 "A Data Transfer Object validates incoming request parameters via `class-validator` before they can reach the database."

Q7 How does backend pagination work?

👉 "We calculate `skip = (page - 1) * limit` and fetch 18 items per page concurrently with a total count query in a single `Promise.all`."

Q8 What indexes did you add to PostgreSQL and why?

👉 "Single indexes on `price` and `createdAt` for sorting, plus a composite index on `(categoryId, price)` to optimize filtered catalog queries."

Q9 How does your search feature work?

👉 "We enabled PostgreSQL's native `pg_trgm` extension and built a GIN trigram index for fast substring fuzzy matching across titles and descriptions."

Q10 How do you handle cloud free-tier cold starts?

👉 "Our frontend fetch client has an `AbortSignal.timeout(35000)` with automatic backoff retry to tolerate Render's ~30-second sleep wake-up time."

Q11 What happens if PostgreSQL goes down?

👉 "In our `PrismaService`, we catch connection failures at startup and initialize a zero-dependency in-memory store so the API never crashes."

Q12 How are API errors formatted?

👉 "A global `HttpExceptionFilter` catches all exceptions and returns a consistent JSON shape: `{ statusCode, message, error, timestamp, path }`."